Data Construction for Fine-Grained Error Typing in Process Reward Models

Qin Yan

This document is the data construction part of a team course project (Process Reward Modeling in LLMs, Heidelberg University, 2026).

1 Introduction

Large language models remain prone to subtle logical and arithmetic errors when producing multi-step solutions, particularly in mathematical reasoning. Process Reward Models (PRMs) address this by scoring each intermediate step of a solution rather than only its final answer, providing a denser training and search signal than an outcome-only verifier. A recent line of work further decomposes this step-level scoring into two stages: a model first classifies the specific type of error present in a step, then uses this classification, together with the original context, to estimate the step's overall correctness. PathFinder-PRM (Pala et al., 2025) instantiates this idea with a two-category error taxonomy (mathematical and consistency errors). Its own ablations report a small, monotonic gain from adding error typing on PRMBench: 64.9 with no error typing (single-stage correctness only), 67.3 with one merged error category, and 67.7 with the two separate categories.

This raises a natural question that the original work does not itself explore: does finer-grained error typing continue to help as the taxonomy is split further, or does the benefit saturate, or even reverse, once categories become too narrow and too noisy to annotate and predict reliably? A coarser taxonomy is easier to label consistently but conflates distinct failure modes. A finer taxonomy separates these failure modes but asks a classifier to make more individually difficult decisions from the same amount of supervision, and some resulting categories may be intrinsically rare or context-dependent. This trade-off bears on how future PRM training pipelines should choose an error taxonomy.

To support this comparison, we construct training data from PRM800K (Lightman et al., 2024) at four label granularities that differ only in how many error categories are predicted before the final correctness judgment. This document describes how this data was constructed. Model training and evaluation are outside its scope.

2 Related Work

Process Reward Models. Process Reward Models (PRMs) score each intermediate step of a multi-step solution, in contrast to Outcome Reward Models, which score only the final answer. Lightman et al. (2024) introduced PRM800K, a large, human-annotated corpus of step-level correctness labels for mathematical solutions, and showed that a PRM trained on this data substantially improves best-of-k selection over an outcome-only verifier. PathFinder-PRM (Pala et al., 2025) extends this line of work with a hierarchical, error-aware design: rather than predicting a single correctness label directly, it first classifies each step's mathematical and consistency errors and then conditions on these predicted labels, together with the original context, to estimate the step's reward. It reports a new state-of-the-art PRMBench score while using substantially less training data than prior discriminative PRMs. Our work takes this hierarchical, error-aware design as a starting point and asks how its benefit changes as the underlying error taxonomy is made coarser or finer, an axis the original paper's own ablations touch on with three settings but do not extend further.

Evaluation Benchmarks. PRMBench (Song et al., 2025) is a benchmark for process-level reward models. It takes correct solutions from both the training and test splits of PRM800K, injects reasoning errors of several distinct types, and scores a PRM separately on each error type. The PathFinder-PRM results cited in Section 1 are reported on PRMBench. Because its questions come from PRM800K, we exclude them from our train-

ing data (Section 3).

3 Data Construction

We construct our training data from PRM800K (Lightman et al., 2024), the human-annotated, step-level dataset also used by the original PathFinder-PRM pipeline (Pala et al., 2025). The construction proceeds in four stages: (i) defining a four-way error taxonomy that further decomposes the two error categories used by the original paper, (ii) annotating candidate steps with this taxonomy using an LLM and filtering the results for consistency with PRM800K's human labels, (iii) selecting a balanced sample from the filtered pool, and (iv) formatting the selected sample into four training-data versions that isolate the effect of label granularity.

3.1 Four-Dimensional Error Taxonomy

PathFinder-PRM's first stage classifies each step along a *Math* axis and a *Consistency* axis before a second stage predicts *Correctness* (Pala et al., 2025). We split each axis into two narrower, binary categories: *Calculation* and *Method* from Math, and *Prior-step consistency* and *Domain consistency* (hereafter *PriorStep* and *Domain*) from Consistency. Nine of the ten defining criteria are taken directly from the original paper's annotation prompt (two Calculation, three Method, two PriorStep, two Domain). One further Domain criterion, which flags a step that directly contradicts a condition explicitly stated in the problem, is added by this work. Five further disambiguation rules (e.g., distinguishing an invalid method from a valid one executed with an arithmetic slip) resolve boundary cases not covered by the original prompt. By default only the single most relevant category is marked, and more than one only when the underlying errors are independent and each sufficient on its own. An abridged version of the annotation prompt is given in Appendix A. *Correctness* is not part of this taxonomy. Following the original paper, it marks whether a step is both error-free and optimal.

3.2 Data Annotation and Consistency Filtering

Following the original pipeline's behavior, only steps with a human rating of -1 (24.15% of all expanded PRM800K records) are sent to an LLM for four-way error labeling. Steps rated 1 (correct and optimal) or 0 (correct but suboptimal) are assigned all five labels directly without any LLM call: all four error categories are set to 1 (no problem), and *Correctness* is set to 1 or 0 respectively. Labels for rating $= -1$ steps were produced by DeepSeek-V4.1-Flash with thinking mode disabled, using a prompt adapted from the taxonomy above (Appendix A) and the same reasoning-then-score response format as the original paper's annotation prompt. A step is discarded as inconsistent with its human rating if the LLM assigns "no problem" to all four categories despite a human rating of -1, after up to two re-annotation retries, following the original pipeline's policy. All label-quality reviews reported below were performed by an AI agent, not by human reviewers.

Of 7,500 labeled rating $= -1$ steps, 5,952 unique (question, prior-steps, current-step) triples remained after deduplication and consistency filtering. On a random sample of 40 records, 90% of labels were judged acceptable. PriorStep was the exception: only 50% of its single-pass "problem present" labels were acceptable (30 reviewed). We therefore added a confirmation step not present in the original pipeline: each PriorStep "problem present" label was re-annotated twice more (once with thinking mode disabled, once enabled) and kept only if at least one re-annotation reproduced it. This reduced such labels from 268 to 116 (97 after deduplication) and raised their acceptable rate to 83% on a fresh sample of 30.

3.3 Sample Selection

The final sample contains 2,330 records, with error and non-error steps balanced 1:1 (1,165 each) instead of PRM800K's natural ratio of about 1:3, so that each error category has enough examples. The original scripts apply no such resampling. The error side includes all 75 *PriorStep* steps, the rarest category, along with 503 *Calculation*, 493 *Method*, and 498 *Domain* steps. A step with several error categories is counted in each. Because PRM800K annotators stop rating a solution at its first error, correct steps tend to appear earlier in a solution than error steps and therefore have shorter contexts. To keep the model from using position or context length as a shortcut, non-error steps are sampled to match the error side's distribution over three bins of context length and three bins of prior-step count.

Because PRMBench's evaluation questions are themselves drawn from PRM800K (Song et al., 2025), questions that overlap with PRMBench are identified by whitespace-normalized text matching and excluded from both the error and non-error candidate pools before sampling. The resulting

2,330-record sample has zero overlap with PRM-Bench's question set.

3.4 Dataset Versions

The selected sample is formatted into four training-data versions that differ only in the granularity of the error labels predicted before *Correctness*. 0-class predicts *Correctness* alone in a single stage, matching the original paper's "w/o Separate Subtask Prediction" ablation. 1-class first predicts one binary *Error* label that merges all four categories, matching its "w/o Separate Error Categories" ablation. 2-class uses the original *Math*/*Consistency* split, which is the paper's main design. 4-class uses our full taxonomy and has no counterpart in the original paper. The original results for the first three designs come from a 7B model trained on about 400K samples and are not directly comparable to our 1.5B, 2,330-sample setting.

Each record yields one training row per stage, giving 4,660 rows for each two-stage version (1-, 2-, and 4-class) and 2,330 rows for 0-class. The label format follows the original scripts, and a worked example is given in Appendix B. For 0-class, we replace the original script's 2-class system prompt with one that asks only for *Correctness*.

4 Limitations

Error labels come from an LLM rather than human experts, and all quality reviews were performed by an AI agent on small samples of 30 to 40 records, so the reported acceptance rates are rough estimates. *PriorStep* is the rarest category, with 75 examples, and its labels required an extra confirmation step to reach acceptable quality. Similar errors are sometimes labeled *Calculation* and sometimes *Method*, so the boundary between the two categories is noisy. The disambiguation rules were designed for consistent automatic labeling and may not match how a human expert would categorize a borderline step. The 1:1 error-to-correct ratio and the length- and step-count-matched sampling also depart from PRM800K's natural distribution, so results may not transfer to a naturally distributed setting.

References

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*.

Tej Deep Pala, Panshul Sharma, Amir Zadeh, Chuan Li, and Soujanya Poria. 2025. Error typing for smarter rewards: Improving process reward models with error-aware hierarchical supervision. *arXiv preprint arXiv:2505.19706*.

Mingyang Song, Zhaochen Su, Xiaoye Qu, Jiawei Zhou, and Yu Cheng. 2025. PRMBench: A fine-grained and challenging benchmark for process-level reward models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*.

A Annotation Prompt

The prompt below is an abridged and condensed version of the one used to obtain the four-way error labels (Section 3.1). It keeps the four criteria and the required output format, and omitted parts are marked in the text. `[CONTEXT]` is replaced by the question, previous steps, and current step, and bracketed items such as `[0 or 1 only]` mark where the model fills in its response.

```
You are an analytical math instructor grading a student's work. Think step-by-step through
your analysis. Below is the math question, previous steps by the student, and current step
to evaluate.

[CONTEXT]

Assign binary scores (0 = problem present, 1 = no problem) based on four criteria:

Calc_ok) Calculation -- Is the current step, on its own, free of arithmetic/algebraic
mistakes? (The operation, formula, or approach is assumed valid; only its execution is in
scope. A valid operation that is merely miscalculated is a Calc_ok problem.)

Method_ok) Method/formula -- Is the current step, on its own, using a valid approach or
formula? (If the operation, formula, or approach itself does not hold, regardless of whether
the arithmetic on top of it is correct, that is a Method_ok problem, not Calc_ok.)

Prior_ok) Consistency with this solution's own previous steps -- Score 0 only when the
current step actually contradicts something this solution has already established; merely
continuing from an earlier mistake is not a Prior_ok problem. Automatically 1 if there are
no previous steps yet.

Domain_ok) Consistency with problem/domain -- Score 0 only for a clear, explicit conflict
with the problem's stated conditions, established facts, or a theorem's applicability
conditions; an invalid method choice alone (Method_ok=0) does not by itself imply Domain_ok=0.

[... bullet-point examples for each criterion, and a restated single-category-by-default
instruction, omitted ...]

You must format your answer as below:

Reasoning:
[Provide detailed analysis, showing all verification steps and explicitly identifying any errors found]

Final answers:
Calc_ok:      [0 or 1 only]
Method_ok:    [0 or 1 only]
Prior_ok:     [0 or 1 only]
Domain_ok:    [0 or 1 only]
```

B Data Example

The example below is a *PriorStep*-error record from the 4-class version, taken from the training data with intermediate solution steps omitted and lines wrapped for readability. It shows the Stage 1 target sequence, which contains all four error labels. In the corresponding input sequence, each label token is replaced by `<extra>`.

```
User:
You are a Math Teacher. Given a question and a student's solution, evaluate the calculation
correctness, method validity, consistency with prior steps, and consistency with the problem
domain of the current step and whether it will lead to the correct final solution

Question: Alice buys three burgers and two sodas for $3.20, and Bill buys two burgers and
a soda for $2.00. How many cents does a burger cost?

Assistant (stage 1, labels):
Let's call the price of a burger x.

And let's call the price of a soda y.

Now we can set up the equations: 3x+2y=320 and 2x+y=200.
[... 9 intermediate solving steps omitted ...]
```

```
So y=40.

So a soda costs 40 cents.

Current Step: And 40 cents is the correct answer.

# Answer

40 Calculation: <+>, Method: <+>, Prior-step consistency: <->, Domain consistency: <+>
```

Stage 2 is identical except that , `Correctness: <->` is appended to the assistant turn.

The step is labeled as a *PriorStep* error because the question asks for the price of a burger, which the solution had already found to be 80 cents, but the final step gives the price of a soda instead. This contradicts the solution's own earlier result and involves no calculation or method error.